

Day 3: How much does the random seed alone move a small transformer's final validation loss?

Version 3.0.6

Research advisor: Claude. Owner: Fan Pu.

September 15, 2026

D1 (spine) · H1.3 · backlog item 3

1 Assignment Overview

Every later day of this sprint will compare two token mixers and say that one has a lower loss than the other by some amount Δ . Whether Δ means anything depends on a number nobody at this scale reports: how far the loss of one fixed architecture moves when nothing changes but the random seed. Today we measure that number for the transformer baseline at three sizes. We train the same model twice per size, changing only the seed, and turn the two final losses into a standard deviation with an honest range around it. The day closes when the 30M pair has finished and its two final validation losses, the spread they imply, and the determinism floor (the largest difference between two runs that share a seed, which measures the GPU's own run-to-run variation rather than the seed's) are in the log; the 60M and 125M pairs run in a queue afterwards and are graded when they land.

The setting. Today's GPU budget inside the working day is about 3 GB10-hours (the GB10 is the GPU of the DGX Spark this sprint runs on): two short smoke runs (a *smoke run* is a short run whose only purpose is to check that the pipeline works) and the two 30M runs. The full six-run queue that the day launches costs about 31 GB10-hours by our estimate (Section 5), which is roughly 4% of the sprint's 720 GPU-hours (30 days at 24 hours). The goal is to measure, as small an interval as two seeds per size allow, the standard deviation of final validation loss across seeds. The constraint that makes the day non-trivial is that two runs give a very rough estimate of a spread, so most of the work is in making a rough number honest: pooling across sizes, stating the range it could be in, and showing that the GPU's own non-determinism and the finite validation set are not what we measured.

What you will implement

1. `seed_stats`, the pooled seed standard deviation with its range and the minimum detectable difference (Section 3)
2. `check_disjoint`, the count of validation windows (fixed-length stretches of held-out text) that appear in training (Section 4)
3. `lr_at`, the warmup-then-cosine learning-rate schedule (Section 4)
4. `ref_nll` and `evaluate`, a naive reference and the batched, token-weighted validation loss (Section 4)

What you will run

1. Data preparation: FineWeb-Edu (a public corpus of educational web text) to sixteen token shards (flat files of token ids), and a look at what is in them (Section 4)
2. Two same-seed smoke runs of 300 steps, which give the determinism floor (Section 7)
3. The six-run queue, 30M/60M/125M at two seeds each; the 30M pair finishes today (Section 7)

What you will write

1. Four pencil-and-paper parts: the noise of a finite validation set, the two-run estimate, steps and hours, and peak memory (Section 2.2, Section 3, Section 5)

2. Predictions, before the first run ([Section 6](#))
3. The experiment log and your call on tomorrow's follow-up ([Section 8](#))
4. The finding in your own words ([Section 9](#))

What you can use

- `fla` (flash-linear-attention, v0.5.2): the model classes. `TransformerForCausalLM` is the attention baseline; it computes the loss itself when given labels.
- `torch`: everything else in the training loop, including `torch.optim.AdamW`, `torch.autocast` (the mixed-precision context), and `torch.nn.functional.scaled_dot_product_attention` (SDPA), the attention function the model runs through. SDPA chooses among several backend kernels; `train.py` pins the one called Flash, which was the fastest on this box in the throughput measurements restated below.
- `datasets` and `tiktoken`: streaming download of FineWeb-Edu and the GPT-2 tokenizer.
- `scipy.stats.chi2` and `numpy` in the statistics; `matplotlib` in the plot.
- Written from scratch, by you: the five functions listed above. They are the pieces whose failure is silent, which is why they are yours: a wrong schedule, a wrong loss average, a wrong overlap count, or a wrong standard deviation all run to completion and hand you a plausible number. Everything whose failure is loud (data loading, the training loop, checkpointing, queueing, logging, plotting) is written for you and tagged [AI].

What the code looks like

The zip is a snapshot of one repository, `testbed`, that lasts the whole sprint; today is its first snapshot. Its layout copies public codebases that many people have worked on for the same job, so that reading the starter code teaches you those codebases: the package-beside-`tests/` layout and `tests/adapters.py` are those of the Stanford CS336 assignment starters, the reference-beside-fast-version pattern is `fla`'s, the flat training script is nanoGPT's (Karpathy's minimal GPT trainer), one TOML file per run (TOML is a plain-text configuration format) is torchtitan's (PyTorch's reference pretraining codebase), and the sweep-as-a-Python-file is Zoology's (the synthetic-recall benchmark codebase of Arora et al.). The README names the counterpart of every file.

1. `testbed/`: the package. `schedule.py`, `data_check.py`, `evals/val_loss.py`, `evals/naive.py`, and `analysis/seed_stats.py` hold one stub each, with the typed signature and docstring from this handout, raising `NotImplementedError`. `model.py` (shapes, config, the SDPA shim), `data.py` (shard format, windows, batch order), and `config.py` (reading a run's TOML file) are complete.
2. `tests/`: one test per function, run as `pytest -k test_<slug>`; `tests/adapters.py` is the glue you fill in so the tests can call your code. Do not edit the tests.
3. `scripts/`: `prepare_data.py`, `look_at_shards.py`, `check_data.py`, `train.py`, `summarize.py`. Each is a flat script that reads a data directory or a run file and takes at most a seed and an output directory besides; no script is edited to set a value.
4. `experiments/day3_seed_variance/`: today's material. One TOML file per run (`attn_30M.toml`, `attn_60M.toml`, `attn_125M.toml`, identical except for the size and the token budget), the smoke preset `attn_30M_smoke.toml`, `sweep.py` (the list of six runs), `run.sh` (runs the list), `log_entry_template.md` (the log skeleton printed in [Section 8](#)), and `sample_runs/` (four synthetic runs in the exact output format, for warming up `summarize.py`).
5. `examples/`: `shard_roundtrip.py` and `simulate_shat.py`, complete and runnable, walked through below.
6. `README.md`: setup, tests, each script's command, expected runtimes on the GB10, and the changelog.
7. `runs/`: created by `train.py`. Each run writes `runs/<mixer>_<size>_s<seed>/` (today `attn_30M_s0` and so on) containing a copy of its config, an environment file, `metrics.jsonl`, and `summary.json` when it completes, so that any number in a run directory is reproducible from that directory alone.

Before you start

The following must exist. First, the environment: `torch 2.14.0+cu130`, `fla 0.5.2`, and `TORCH_CUDA_ARCH_LIST="12.1a"` exported, which is the environment in which `fla`'s kernels were verified on this box. Second, three throughput measurements taken on this box with this model code, a batch of 32 windows of 1024 tokens, `bf16` (16-bit brain floating point, the `matmul` precision used throughout), and a 32,000-symbol vocabulary, which are restated here because [Section 5](#) uses them: the attention baseline trains at 77,546 tokens per second at 30M ($d = 512$, $L = 10$, 34.1M non-embedding parameters, peak memory 9.3 GB), 59,511 at 60M ($d = 768$, $L = 9$, 63.7M), and 32,926 at 125M ($d = 768$, $L = 18$, 127.4 M). Third, the starter zip unpacked and `uv sync` run (`uv` is the Python package manager the README uses); the tests need nothing beyond that. Nothing else from earlier days is needed.

Kill and pivot criteria

If a smoke run's validation loss at step 0 is more than 0.05 nats (the unit of cross-entropy loss under a natural logarithm; [Section 2.1](#)) from the value you predict in [Section 6](#), or its training loss at step 300 is above 7.5, the queue is not started: the pipeline is broken, and today's result is the diagnosis. If the two same-seed smoke runs differ in training loss at step 300 by more than 0.02 nats, a seed pair would measure the GPU rather than the seed; the queue is still launched, since its numbers are needed either way, but the day's question changes to how large the GB10's non-determinism floor is and what removing it costs, and that floor is the result you log.

Low-Resource Tip: If things go slowly

If hour three arrives and the queue has not launched, the reduced result that still closes the day is: the training script runs at 30M for 300 steps on real data, the step-0 validation loss matches the prediction, and the same-seed determinism floor is measured. That is Problem (`smoke_floor`) alone. Launch the queue before you stop, even late, so that the 30M pair is graded first thing tomorrow. Concrete settings: nothing changes in `experiments/day3_seed_variance/`; you skip Problems (`simulate_shat`) and (`check_disjoint`) and run them tomorrow while the queue is busy.

Who uses this

The first matched comparison of a linear-attention mixer against this baseline, at the same budgets, cannot be written up without today's number: its central sentence is "the gap is (or is not) larger than the seed band", the band being the spread of final losses across seeds that today measures. Every later comparison across mixers, gate variants, and hybrid layouts draws the band as a shaded region on its plots. The optimizer study that tests whether the transformer's learning rate is also the right learning rate for other mixers uses today's fixed optimizer settings as its control arm. The study of what the recurrent state stores needs the checkpoints that `train.py` writes at every 10% of training. And `train.py` itself, unchanged, is the training loop for every run in the sprint.

2 The training run at a glance

Today's experiment is six training runs and one subtraction. Before any component, here is one run from top to bottom, with every tensor's shape stated in words. FineWeb-Edu text is tokenized with the GPT-2 tokenizer into flat streams of token ids stored as unsigned 16-bit integers, 100M tokens per file (a *shard*); shard 0 is the validation set and shards 1 to 15 are training data. A training *window* is $T = 1024$ consecutive tokens taken from a shard at a multiple of T . A batch is $B = 32$ windows, an integer tensor of shape (32, 1024). The model is a transformer with width d and L layers (three sizes today, [Table 2](#)) and a vocabulary of $V = 50,304$ symbols; it maps the batch to logits of shape (32, 1024, 50,304) in `bf16`, and the

loss is one fp32 scalar, the mean over the $32 \cdot 1023$ next-token predictions of the negative log-likelihood of the true next token. One AdamW step follows at the learning rate the schedule gives for that step, and this repeats for S steps, where S is the token budget divided by $BT = 32,768$ tokens per step. Every 5% of the run the model is evaluated on a fixed set of 4096 validation windows (shape (4096, 1024)), and every 10% a checkpoint is written; at the end a larger evaluation on 16,384 windows produces the run’s final validation loss, in `summary.json`.

Six runs: three sizes at two seeds each, seeds 0 and 1. For each size, the two final validation losses differ by some amount. That amount, turned into a standard deviation and pooled across the three sizes, is the day’s result. Everything in this handout is in service of making that one number trustworthy.

What the seed does, precisely: it sets the initial weights, and it sets the order in which the training windows are shown. It does not change *which* windows are used; a budget of D tokens always uses the first D/T aligned windows of the training shards in file order, and the seed permutes them (Section 4 says why). So “seed” today means “initialization plus data order”, with the data itself held fixed. What the seed does not control is the GPU’s own run-to-run variation, which Section 2.2 measures separately.

2.1 Remark: conventions

A *nat* is the unit of cross-entropy loss when the logarithm is natural; $\ln 2 = 0.693$ nats is one bit, and since perplexity is e^L , a difference of 0.01 nats is about a 1% difference in perplexity. *Validation loss* is the mean next-token negative log-likelihood (NLL) over held-out windows the model never trains on, in nats. A *step* is one gradient update on one batch of $BT = 32,768$ tokens, and the *token budget* D of a run is $S \cdot BT$ where S is its number of steps. A *shard* is a flat one-dimensional stream of uint16 token ids on disk; an *aligned window* is $T = 1024$ consecutive tokens starting at a multiple of T within one shard, so windows never overlap and never cross a shard boundary. Validation windows are the first 4096 aligned windows of shard 0 for the periodic evaluations and the first 16,384 for the final evaluation; 2^{22} and 2^{24} tokens respectively. “Seed standard deviation” means the standard deviation, across seeds, of the *final* validation loss under the fixed configuration of Table 3; it is written σ when it means the true value and $\hat{\sigma}$ when it means an estimate from data. All of these are the values `train.py` uses; none is changed today.

2.2 Why two runs differ

Fix the architecture, the hyperparameters, and the data. Run twice. The final losses will differ, and there are three reasons why, only one of which we want.

1. **The seed.** Different initial weights and a different batch order. This is the quantity of interest.
2. **The GPU.** Some kernels sum floating-point numbers in a different order on each launch, and floating-point addition is not associative, so two runs with the *same* seed can drift apart. We call the size of that drift the *determinism floor*: the largest difference in training loss between two same-seed runs over their first 300 steps, measured in Problem (smoke_floor). PyTorch’s documentation states that the fused attention backends, including the Flash backend the model uses, run non-deterministic backward code by default [PyTorch contributors, 2026, §Avoiding nondeterministic algorithms], so the floor is probably not zero.
3. **The validation set is finite.** Validation loss is an average over N tokens, and averages over finite samples have noise. We call this the *evaluation noise*: the standard deviation the validation loss would show if the validation windows were redrawn.

The plan is to measure the total from a two-seed pair and to show that the second and third sources are small compared with it, so that the total is really the first. We do not try to split the seed’s effect into “initialization” and “data order” today; that is one of the follow-ups in Problem (your_call).

Example (eval_noise): The mechanism behind evaluation noise

Suppose each token's loss were an independent draw with standard deviation s_{tok} . The mean of N such draws has standard deviation $s_{\text{tok}}/\sqrt{N}$. With $s_{\text{tok}} = 2.5$ and $N = 100$ tokens the noise is $2.5/10 = 0.25$ nats; with $N = 10,000$ it is $2.5/100 = 0.025$ nats. Tokens inside one document are not independent (the second half of a sentence is easier given the first), so a conservative correction is to count only half of them: with $N = 10,000$ the corrected noise is $2.5/\sqrt{5000} = 0.035$ nats.

Problem (eval_noise): Is the validation set big enough? (0.5 points) [you]

- (a) Per-token loss on web text has a standard deviation across tokens of roughly 2 to 3 nats; this is a rough empirical figure, not something we derive, so take 2.5. The periodic evaluations use $N = 2^{22}$ tokens and the final evaluation $N = 2^{24}$. Using the half-independence correction from Example (eval_noise), compute the evaluation noise for both.

Deliverable: Two numbers in nats, each with one line of arithmetic.

- (b) Both seeds at a size are evaluated on the *same* fixed windows. Does that make the evaluation noise in the difference $x_1 - x_2$ of their final losses larger, smaller, or the same as the noise in x_1 alone?

Deliverable: One sentence with the reason.

3 Estimating a spread from two numbers

Two final losses per size is what the budget allows, and two numbers make a very rough estimate of a spread. This section builds, from the two-point case up, the estimator `summarize.py` reports and the range that must always be quoted next to it.

3.1 The two-run estimate

You will have two final losses at a size, x_1 and x_2 . The sample standard deviation is the square root of the sum of squared deviations from the mean divided by $n - 1$. With two points the mean is the midpoint, each point is $|x_1 - x_2| / 2$ from it, the two squared deviations sum to $(x_1 - x_2)^2 / 2$, and $n - 1 = 1$:

$$\hat{s} = \frac{|x_1 - x_2|}{\sqrt{2}}. \quad (1)$$

Example (two_run): One pair

$x_1 = 4.41$, $x_2 = 4.43$. Then $\hat{s} = 0.02/\sqrt{2} = 0.0141$ nats.

3.2 How rough that is

Model each final loss as $x = \mu + \sigma\varepsilon$, where μ is the typical loss at this size, σ is the true spread we want, and ε is a standard normal random number (mean 0, standard deviation 1) that the seed determines.

Then $x_1 - x_2 = \sigma(\varepsilon_1 - \varepsilon_2)$. The difference of two independent standard normals is normal with variance $1 + 1 = 2$, so $x_1 - x_2 = \sigma\sqrt{2}Z$ for a standard normal Z , and by [Equation 1](#)

$$\hat{s} = \sigma |Z|. \quad (2)$$

The estimate is the truth times the absolute value of one standard normal draw (a quantity with a name, the *half-normal* distribution). Before looking up what that implies, watch it. The following is `examples/simulate_shat.py`:

```
import numpy as np

rng = np.random.default_rng(0)
sigma = 1.0
x1, x2 = rng.normal(0, sigma, 10_000), rng.normal(0, sigma, 10_000)
s_hat = np.abs(x1 - x2) / np.sqrt(2)
p10, p90 = np.percentile(s_hat, [10, 90])
print(f"true sigma = {sigma}")
print(f"10th percentile of s_hat = {p10:.3f}, 90th = {p90:.3f} (ratio {p90/p10:.1f}x)")
print(f"fraction of pairs with s_hat < 0.5 sigma: {(s_hat < 0.5).mean():.3f}; > 1.5 sigma: {(s_hat > 1.5).mean():.3f}")
```

Problem (`simulate_shat`): Watch a two-run estimate wobble (0.5 points) [you]

- (a) Run python `examples/simulate_shat.py`. Then change σ to 0.015 and run it again.

Deliverable: The 10th and 90th percentiles of $\hat{s}$ and the fraction of pairs with $\hat{s} < 0.5\sigma$ or $\hat{s} > 1.5\sigma$, for both values of σ , and one sentence on what changed and what did not.

- (b) If the 30M pair tonight gives $\hat{s} = 0.012$, what range of true σ is consistent with it?

Deliverable: One sentence with a range.

The theory behind what the toy shows: $|Z|$ is below 0.126 in 10% of draws and above 1.645 in another 10%. (These come from the normal distribution's inverse: $P(|Z| < c) = 0.1$ means $P(Z < c) = 0.55$, and `scipy.stats.norm.ppf(0.55)` is 0.126; likewise `ppf(0.95)` is 1.645.) So 80% of the time, $\hat{s}$ lands between 0.13σ and 1.64σ , a factor of 13 between the ends. One pair tells you the order of magnitude of σ , not its value.

3.3 Pooling across sizes

Assume for today that the spread is the same at 30M, 60M, and 125M. This is an assumption, not a fact: larger models plausibly wobble less, and with two seeds per size you cannot test it ([Section 8](#) says how to handle that). Under the assumption, the three pairs are three independent estimates of one σ , and combining them beats trusting any one. Add up every size's sum of squared deviations, divide by the total *degrees of freedom* ν , which is the number of independent deviations, $n_i - 1$ per size, so $\nu = 3$ for three pairs, and take the root:

$$\hat{s}_{\text{pooled}}^2 = \frac{\sum_{i \in \text{sizes}} \sum_{j \in \text{seeds}} (x_{ij} - \bar{x}_i)^2}{\sum_i (n_i - 1)}, \quad \text{which for pairs is } \frac{\sum_i (x_{i1} - x_{i2})^2 / 2}{3}. \quad (3)$$

By the same reasoning as [Equation 2](#), $\hat{s}_{\text{pooled}}^2 / \sigma^2 = (Z_1^2 + Z_2^2 + Z_3^2) / 3$, the average of three squared standard normals, and averages fluctuate less than single draws. The distribution of a sum of ν squared standard normals is the *chi-square distribution with ν degrees of freedom*, and

`scipy.stats.chi2.ppf([0.1, 0.9], nu)` returns its 10th and 90th percentiles. Dividing by ν and taking roots gives [Table 1](#).

What you have	ν	80% range of $\hat{s}/\sigma$	High-to-low factor
one size, 2 seeds	1	0.13 to 1.64	$13 \times$
three sizes, 2 seeds (pooled)	3	0.44 to 1.44	$3.3 \times$
three sizes, 3 seeds (pooled)	6	0.61 to 1.33	$2.2 \times$

Table 1: How far a pooled estimate can be from the truth, from the chi-square percentiles.

Turned around: given $\hat{s}_{\text{pooled}}$ with $\nu = 3$, the true σ is probably between $\hat{s}_{\text{pooled}}/1.44$ and $\hat{s}_{\text{pooled}}/0.44$. We call that the 80% *range*: the interval that contains the truth in 80% of repetitions. It is reported next to the number, always.

Example (pooled): Three pairs

Final losses 30M : [4.41, 4.43], 60M : [3.90, 3.88], 125M : [3.50, 3.53]. Squared-deviation sums per size, $(x_{i1} - x_{i2})^2/2$: 0.0004/2 = 0.0002, 0.0002, and 0.0009/2 = 0.00045. Total 0.00085, $\nu = 3$, so $\hat{s}_{\text{pooled}} = \sqrt{0.00085/3} = 0.01683$ nats. Range: $0.01683/1.4435 = 0.01166$ to $0.01683/0.4414 = 0.03814$ nats.

3.4 The smallest gap worth believing

The next use of the number is a comparison of two mixers with two seeds each. The mean of two runs has standard deviation $\sigma/\sqrt{2}$; the difference of two such means has standard deviation $\sqrt{\sigma^2/2 + \sigma^2/2} = \sigma$. A gap of 1σ happens by chance often; a gap of 2σ arises by luck about 5% of the time (the two-sided normal tail beyond 2), which is the conventional “probably real” line. Our $\hat{s}$ could itself be $0.7 \times$ too small (the low end of the $\nu = 3$ row of [Table 1](#)), so we are stricter:

$$\text{MDD}_{n=2} \approx 2.5\hat{s}_{\text{pooled}}. \quad (4)$$

MDD is the *minimum detectable difference*: with two seeds per arm, a loss gap smaller than it is written up as “not detected”, never as “no difference”.

Example (mdd): From the pooled example

From Example (pooled), $\text{MDD}_{n=2} = 2.5 \times 0.01683 = 0.0421$ nats.

Problem (seed_stats): The number the rest of the sprint uses (1 point) [you]

Implement `seed_stats` in `testbed/analysis/seed_stats.py`. It takes the final validation losses grouped by size and returns the pooled estimate of [Equation 3](#), its degrees of freedom, the 80% range from the chi-square percentiles, and the MDD of [Equation 4](#). It is the function `summarize.py` calls, so every seed band reported in the sprint comes from it.

```
def seed_stats(losses_by_size: dict[str, list[float]]) -> dict
```

Input

- **losses_by_size:** `dict[str, list[float]]` Size label to the list of final validation losses (nats) of the seeds at that size. A size with fewer than two entries contributes nothing and is skipped.

Output

- **result:** `dict` Keys `s_pooled` (nats), `nu` (int), `lo` and `hi` (the 80% range, nats), and `mdd` (nats): $\hat{s}_{\text{pooled}}$, $\nu = \sum_i (n_i - 1)$, $\hat{s}_{\text{pooled}} / \sqrt{\chi_{0.9, \nu}^2 / \nu}$, $\hat{s}_{\text{pooled}} / \sqrt{\chi_{0.1, \nu}^2 / \nu}$, and $2.5 \hat{s}_{\text{pooled}}$, where $\chi_{p, \nu}^2$ is `scipy.stats.chi2.ppf(p, nu)`.
- (a) Implement the function. The test checks Example (`pooled`) and Example (`mdd`) to four decimals, the single-size case of Example (`two_run`) with $\nu = 1$, a three-seed case with $\nu = 4$, and that a size with one entry is skipped.

To test your implementation, implement the adapter `[adapters.run_seed_stats]`. Then, run `pytest -k test_seed_stats` and make sure your implementation passes.

Deliverable: The function, the passing test, and the printed dictionary on the input of Example (`pooled`).

4 The training run, piece by piece

[Section 2](#) gave the run as a whole. The pieces that can go silently wrong are the data (is validation text in the training set?), the target the loss is computed against, the schedule, and the evaluation average. Each is built here in the order the run uses it, with the test that checks it. Every choice in this section is held fixed across sizes and seeds today, and is what the first linear-attention comparison will train with; the direction that questions these choices is the optimizer study, not today.

4.1 Data and tokens

The corpus is `HuggingFaceFW/fineweb-edu`, configuration `sample-10BT`, streamed and tokenized with the GPT-2 tokenizer (tiktoken encoding `gpt2`, 50,257 symbols; the final linear layer that maps hidden states to vocabulary logits, the “LM head”, is padded to $V = 50,304 = 786 \times 64$, a multiple of 64 for the matmul). Each document is written as the end-of-text token `EOT = 50256` followed by its tokens. A shard holds 100M tokens in the binary format of Karpathy’s `llm.c` [\[A. Karpathy, 2024\]](#): a header of 256 int32 values (magic 20240520, version 1, token count, zeros) followed by the uint16 tokens. Shard 0 is validation and shards 1 to 15 are training. The format is `llm.c`’s `edu_fineweb10B` layout, so published `llm.c` and nanoGPT curves on the same data can be compared with a file drop later.

Why the vocabulary is larger than in the throughput measurements. The throughput numbers restated in the overview were taken with a 32,000-symbol vocabulary. The LM head is a $d \times V$ matrix, so its cost per token is proportional to V . At 30M the head grows from $32,000 \times 512 = 16.4\text{M}$ to $50,304 \times 512 = 25.8\text{M}$ parameters; with a 34.1M body, the matmul parameters go from 50.5M to 59.9M, a factor of 1.19; at 60M and 125M the factors are 1.16 and 1.09. Expect tokens per second to fall by less than these factors, since the head is the most efficient matmul in the model; [Section 5](#) uses a correction of 0.88, 0.90, and 0.92 at the three sizes, which is our estimate and is checked against the measurement in Problem (`smoke_floor`).

Same windows, different order. Both seeds at a size see the same tokens; only the order differs. If each seed saw a different random slice of the corpus, part of the spread would come from “which text did I train on”, which is not something a paper’s one-run-per-model comparison varies. `testbed/data.py` does this: a budget of D tokens uses the first D/T windows of the training shards in file order, and `batch_order(seed, n)` permutes them with `numpy`’s generator seeded by the seed alone, so a resumed run replays the same order. One consequence: changing the budget changes which tokens are used, so the

`training.budget_tokens` value in each run's TOML file is fixed in Problem (budgets) before the queue starts and not touched afterwards.

Before looking at real data, here is the format on a toy, `examples/shard_roundtrip.py`, which writes three three-token “documents” to a shard, prints the header, reads the stream back through a memory map (`numpy.memmap`, which reads bytes from disk on access instead of loading the file), and cuts it into aligned windows of $T = 4$:

```
$ python examples/shard_roundtrip.py
stream of 13 tokens: [50256, 11, 12, 13, 50256, 21, 22, 50256, 31, 32, 33, 34, 35]
header[:3] = [20240520, 1, 13] (magic, version, token count); file size = 1050 bytes = 1024 +
↳ 2 * 13
read back: [50256, 11, 12, 13, 50256, 21, 22, 50256, 31, 32, 33, 34, 35] dtype uint16
aligned windows of T = 4 : 3 (the tail of 1 tokens is dropped)
[[50256, 11, 12, 13], [50256, 21, 22, 50256], [31, 32, 33, 34]]
seed 0 order: [2, 0, 1] seed 1 order: [0, 1, 2]
window 2 by id: [31, 32, 33, 34]
```

Note that a window can contain an EOT in its middle and can start in the middle of a document; the model learns to treat EOT as a boundary, and the causal mask does not reset there. This is the standard choice and `llm.c`'s.

Problem (look_at_shards): Prepare the data and look at it (0.5 points) [you]

Resource requirements: ≤ 40 minutes of CPU and network, no GPU; 3.2 GB of disk for the sixteen shards

- (a) Launch data preparation in the background and, when it finishes, look at what it wrote:

```
nohup python scripts/prepare_data.py --out data/fineweb_edu --num-shards 16 >
prep.log 2>&1 &
python scripts/look_at_shards.py --data data/fineweb_edu --stats
```

The second command prints the header of the first training shard, its first twelve token ids, the first two aligned windows decoded to text (the first two records of the file), ten random windows from random training shards with the number of document starts in each, and on shard 0 the document count and length statistics.

Deliverable: The header line, the two decoded records, and three sentences: what kind of text this is, how many documents a typical window spans, and anything that looks wrong (non-English text, boilerplate, binary junk).

If validation text also appears in the training shards, every later “generalization” number is off by an amount nobody will notice. The check is to fingerprint every aligned training window (a hash of its T tokens), put the fingerprints in a set, and count how many validation windows hit. About $1.5\text{B} / 1024 = 1.46\text{M}$ fingerprints, a few seconds. Aligned-window matching misses a duplicate document that starts at a different offset; that is a known weakness of today's check, and a rolling hash over all offsets is a follow-up for Problem (your_call).

Problem (check_disjoint): Count validation windows that appear in training (0.5 points) [you]

Resource requirements: ≤ 1 minute of CPU on the real shards

```
def check_disjoint(train_shards: list[np.ndarray], val: np.ndarray, T: int) -> int
```

Input

- **train_shards:** `list[np.ndarray]` One-dimensional uint16 token streams; `numpy.memmap` objects are fine.
- **val:** `np.ndarray` int64 of shape (N, T) , the validation windows.
- **T:** `int` The window length.

Output

- **count:** `int` The number of validation windows equal, token for token, to some aligned training window `shard[k*T:(k+1)*T]`.

- (a) Implement the function in `testbed/data_check.py`. The test plants two aligned copies and one misaligned copy of validation windows in fake shards and expects 2.

To test your implementation, implement the adapter `[adapters.run_check_disjoint]`. Then, run `pytest -k test_check_disjoint` and make sure your implementation passes.

Deliverable: The function and the passing test.

- (b) Run it on the real shards, against the 16,384 windows of the final evaluation set: `python scripts/check_data.py --data data/fineweb_edu`.

Deliverable: The printed count and the number of training windows fingerprinted. Expected: 0 hits. If it is not 0, the sentence in the log that says so.

4.2 The label shift

The model predicts the next token. If a window is `[the, cat, sat, down]`, the target at position 0 is `cat`, at position 1 is `sat`, at position 2 is `down`, and position 3 has no target. A window of T tokens therefore yields $T - 1$ predictions. Somebody has to perform that shift, and the silent failures are doing it twice or not at all.

Example (who_shifts): Who performs the shift

`fla`'s `TransformerForCausalLM.forward(input_ids, labels=input_ids)` shifts internally: it drops the first label, pads the end with `ignore_index` (the label value that PyTorch's cross-entropy skips, `-100`), and returns the mean loss over the $T - 1$ valid positions. Shift *again* yourself, by passing `input_ids[:, :-1]` and `labels=input_ids[:, 1:]`, and the targets become two tokens ahead: the loss plateaus high, around 7. Shift *nowhere*, by computing the loss of the logits at position t against the token at position t , and the target is the input: the loss falls toward 0.

Problem (label_shift): Trace the shift by hand (0.5 points) [you]

- (a) Open `fla/models/transformer/modeling_transformer.py` in your installed v0.5.2 (`python -c "import fla.models.transformer.modeling_transformer as m; print(m.__file__)"`) and find the line that builds the shifted labels; search for `ignore_index`.

Deliverable: The line and its line number.

- (b) With `input_ids = [[5, 7, 9, 2]]`, a single window with $T = 4$, write the label tensor that line produces and the number of positions that contribute to the loss.

Deliverable: A length-4 list and one integer.

4.3 Optimizer and schedule

The optimizer and its settings are given, not derived: they are nanoGPT’s settings for a 124M model, which are safe at 30M and about right at 125M, and deriving them would take the whole day. One value at every size, on purpose: if we tuned the learning rate per size today, the first cross-mixer comparison would depend on how well each mixer happened to be tuned, which is the optimizer study’s question.

Optimizer AdamW, $\beta = (0.9, 0.95)$. The standard choice for language-model pretraining; $\beta_2 = 0.95$ rather than 0.999 so that the second-moment estimate forgets faster, which is what nanoGPT and llm.c use at this scale.

Weight decay 0.1. Applied to every parameter with two or more dimensions (matrices and embeddings) and 0 to the norm gains, the usual split.

Peak learning rate $\eta = 6 \times 10^{-4}$. nanoGPT’s value for 124M, kept at every size (see above).

Warmup 3% of the steps. At step 0 the gradients are large and the Adam moment estimates are empty, so a full-size learning rate can throw the weights somewhere bad in the first hundred steps; *warmup* ramps the rate up linearly.

Decay cosine to 0.1η . Near the end, a large learning rate keeps the weights bouncing around the minimum; *cosine decay* lowers it smoothly so the run settles. The decay is what makes two runs converge toward each other late in training, and it is why the band measured today is tied to this schedule (Section 9).

Gradient clipping global norm 1.0. Caps the update size on the rare step with a huge gradient.

Precision bf16 autocast, fp32 master weights. `torch.autocast` runs matmuls in bf16 while the weights, gradients, and optimizer state stay fp32; the same as the throughput measurements. No `torch.compile`, also so that tokens per second stay comparable.

Batch $B = 32$, $T = 1024$. The throughput measurements’ shape; 32,768 tokens per step.

The schedule, precisely. S total steps, indexed $s = 0, \dots, S - 1$. Warmup length $W = \max(1, \text{round}(0.03S))$. For $s < W$, $\text{lr}(s) = \eta(s + 1)/W$, so the last warmup step $s = W - 1$ is at η . For $s \geq W$, the progress $p = (s - W)/(S - 1 - W)$ runs from 0 to 1 and

$$\text{lr}(s) = \eta \left[0.1 + 0.9 \cdot \frac{1}{2} (1 + \cos \pi p) \right], \quad (5)$$

which is η at $p = 0$ and 0.1η at $p = 1$.

Example (schedule): A schedule you can check by hand

$S = 801$, chosen so that the quarter point of the decay is an integer step. $W = \text{round}(24.03) = 24$. Then $\text{lr}(0) = \eta/24 = 2.5 \times 10^{-5}$; $\text{lr}(23) = \eta$; $\text{lr}(24) = \eta$ ($p = 0$); at $s = 24 + 194 = 218$, $p = 0.25$, $\cos(\pi/4) = 0.7071$, so $\text{lr} = \eta[0.1 + 0.9 \times 0.8536] = 0.868\eta = 5.209 \times 10^{-4}$; at $s = 412$, $p = 0.5$, $\text{lr} = 0.55\eta = 3.3 \times 10^{-4}$; $\text{lr}(800) = 0.1\eta = 6 \times 10^{-5}$. A straight-line decay would give 0.775η at the quarter point, which is how the test tells the two shapes apart.

Problem (lr_at): The schedule (0.5 points) [you]

```
def lr_at(step: int, total_steps: int, peak_lr: float = 6e-4, warmup_frac: float = 0.03,
          final_frac: float = 0.1) -> float
```

Input

- **step: int** The 0-indexed batch the update consumes.
- **total_steps: int** S . In a smoke run of `train.py` this is still the full budget's S , so the smoke run is the exact prefix of the real run.
- **peak_lr, warmup_frac, final_frac: float** η , the warmup fraction, and the floor fraction; `train.py` reads them from the `optimizer.lr`, `lr_scheduler.warmup_frac`, and `lr_scheduler.min_lr_factor` keys of the run's TOML file.

Output

- **lr: float** $lr(s)$ as defined above.
- (a) Implement the function in `testbed/schedule.py`. The test checks Example (schedule), the two monotonicity constraints, and the warmup length of today's 30M run ($S = 9155$, $W = 275$). To test your implementation, implement the adapter `[adapters.run_lr_at]`. Then, run `pytest -k test_lr_at` and make sure your implementation passes.

Deliverable: The function and the passing test.

- (b) Plot `[lr_at(s, 10000) for s in range(10000)]` once and look at it; the test checks a handful of points, your eyes check the rest.

Deliverable: The plot, any format.

4.4 Evaluation

Validation loss is the mean next-token NLL over the fixed validation windows. The windows are fed in batches of B ; each batch returns its own mean over its own $B_{\text{batch}}(T - 1)$ positions. If the number of windows is not a multiple of B , the last batch is short, and averaging the batch means gives it too much weight. The fix is to keep two running sums, batch mean times positions in the batch and positions in the batch, and divide at the end. Today 4096 and 16,384 windows are both multiples of 32, so it will not bite; Problem (evaluate) asks for the weighted version anyway, because the recall evaluation that follows in the sprint will not be so tidy.

Example (token_weighting): Why batch means are not averaged

Two batches: batch 1 has 60 positions with mean loss 4.0; batch 2 has 15 positions with mean loss 5.0. Unweighted mean of the two means: 4.5. Token-weighted: $(60 \times 4.0 + 15 \times 5.0)/75 = 4.2$. The correct answer is 4.2.

The fast version calls the model with labels and lets it shift (Section 4.2). To test it we need a reference that does not inherit the model's shift, and that is the naive version: one window at a time, no labels, the shift done by hand.

Example (ref_on_toy): The reference on a three-token window

Vocabulary $V = 3$; a bigram model whose logits for input token i are row i of $W = \text{diag}(1, 2, 3)$. Window $x = [0, 1, 2]$. Position 0: input 0, target 1, logits $[1, 0, 0]$, $\text{NLL} = \ln(e^1 + 2) - 0 = 1.5514$. Position 1: input 1, target 2, logits $[0, 2, 0]$, $\text{NLL} = \ln(e^2 + 2) - 0 = 2.2395$. Position 2: no target. Mean = 1.8955 nats. The test checks your reference against this number.

Problem (evaluate): Token-weighted validation loss, with a naive reference (1 point) [you]

```
def ref_nll(model, val: torch.Tensor) -> float
```

Input

- **model** Any module whose `forward(input_ids)` returns an object with `.logits` of shape $(1, T, V)$.
- **val:** `torch.Tensor` int64 of shape (N, T) .

Output

- **nll:** `float` For every window w and every position $t \in 0..T-2$: logits = `model(input_ids=w[None]).logits[0, t]` (no labels passed), `nll = logsumexp(logits) - logits[w[t+1]]` in fp32; the mean over the $N(T-1)$ values.

```
@torch.no_grad() def evaluate(model, val: torch.Tensor, B: int, ctx) -> float
```

Input

- **model** The fla model, or the test's stand-in; `forward(input_ids, labels, use_cache)` returns an object with `.loss`.
- **val:** `torch.Tensor` int64 of shape (N, T) ; fed in batches of B along the first axis.
- **B:** `int` The batch size.
- **ctx** A context manager the forward pass runs inside: `train.py` passes the SDPA Flash context, the test passes `contextlib.nullcontext()`.

Output

- **loss:** `float` The token-weighted mean NLL in nats. The forward call is exactly the training loop's: inside `ctx` and `torch.autocast(device_type=val.device.type, dtype=torch.bfloat16, enabled=val.is_cuda)`, `model(input_ids=x, labels=x, use_cache=False).loss`. Weight each batch by $B_{\text{batch}}(T-1)$; `model.eval()` on entry, `model.train()` before returning.
- (a) Write `ref_nll` in `testbed/evals/naive.py`, beside the fast version, which is where `fla` keeps its reference recurrences. This is the slow, unbatched version, and it is the reference precisely because it does the shift by hand.

To test your implementation, implement the adapter `[adapters.run_ref_nll]`. Then, run `pytest -k test_ref_nll` and make sure your implementation passes.

Deliverable: The function and the passing test against Example (`ref_on_toy`).

- (b) Write `evaluate` in `testbed/evals/val_loss.py`. The test builds a tiny stand-in model on CPU whose `forward(input_ids, labels=...)` shifts exactly as `fla` does, feeds 5 windows with $B = 2$ so the last batch is short, and requires `evaluate` to match your `ref_nll` to 10^{-5} . No data files and no GPU are needed.

To test your implementation, implement the adapter `[adapters.run_evaluate]`. Then, run `pytest -k test_evaluate` and make sure your implementation passes.

Deliverable: The function and the passing test.

5 Accounting

The pieces are built; before running them we work out what the runs cost, so that a measured number that disagrees with the estimate is noticed. Run time is $D/(\text{tokens per second})$. The budgets are given, not derived: $D = 300\text{M}$, 600M , and 1.2B tokens for 30M, 60M, and 125M, about 9 tokens per non-embedding parameter, under half the compute-optimal rule of 20 tokens per parameter of Hoffmann et al. [J. Hoffmann et al., 2022, §3], chosen so that the three-size ladder fits the sprint’s GPU time. The ratio 1 : 2 : 4 is what matters for a trend; the tokens-per-parameter number belongs in every write-up.

Size	d	L	Heads	Body params	Embedding + head	Budget D
30M	512	10	8	34.1M	$2 \times 25.8\text{M}$	300M
60M	768	9	12	63.7M	$2 \times 38.6\text{M}$	600M
125M	768	18	12	127.4M	$2 \times 38.6\text{M}$	1.2B

Table 2: The three configurations. Head dimension 64 at every size; the input embedding and the LM head are separate (untied) $V \times d$ matrices with $V = 50,304$. Body counts are the throughput measurements’ values; the shapes are the presets in `testbed/model.py`, the budgets the `training.budget_tokens` key of the three run files in `experiments/day3_seed_variance/`.

Setting	Value
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices, 0 on norm gains, fused
Learning rate	peak 6×10^{-4} , 3% linear warmup, cosine to $0.1 \times \text{peak}$
Batch	$B = 32$ windows of $T = 1024$ tokens; 32,768 tokens per step
Precision	bf16 autocast, fp32 master weights; no <code>torch.compile</code>
Clipping	global gradient norm 1.0
Evaluation	every 5% of S on 4096 windows; final on 16,384 windows of shard 0
Checkpoints	weights (bf16) every 10% of S ; full state for resuming at the same points
Seeds	0 and 1; the seed sets the initialization and the window order, not the window set

Table 3: The fixed configuration, as `experiments/day3_seed_variance/attn_30M.toml` states it; the 60M and 125M files differ only in the size and the budget. Nothing in it varies across today’s six runs.

Example (hours): One run's steps and hours

At 60M, $D = 600\text{M}$ tokens and $BT = 32,768$ tokens per step: $S = \lfloor 600\text{M} / 32,768 \rfloor = 18,310$ steps. At the measured 59,511 tokens per second the run takes $600\text{M} / 59,511 = 10,082 \text{ s} = 2.80 \text{ h}$; with the larger head costing 10% of throughput (the 0.90 correction of [Section 4.1](#)), 3.11 h; two seeds, 6.2 h.

Problem (budgets): Steps, hours, memory, and when the GPU frees up (0.5 points) [you]

- (a) For each size, compute S and the wall-clock per run from the throughput measurements restated in the overview and the corrections 0.88, 0.90, 0.92. Sum for the six-run queue.

Deliverable: A table with columns size, D , S , corrected tokens per second, hours per run, hours per pair; a total; and the clock time at which the queue finishes if launched at hour 2.5 today.

- (b) Apply two rules: the 30M pair must finish inside today’s working day, at most 3 h after launch, and the whole queue must finish within 36 GPU-hours of launch, so that tomorrow is an analysis day and the day after gets the GPU back. If either rule fails, scale all three budgets by the same factor, keeping 1 : 2 : 4, and write the new values into the `training.budget_tokens` key of the three run files.

Deliverable: The budgets you will actually use, and one sentence justifying them.

- (c) The first linear-attention comparison trains a gated delta-rule mixer at the same budgets, and in the same throughput measurements that mixer ran at $0.64 \times$ the attention baseline’s tokens per second. How many GPU-hours is its 125M pair, and what does that imply for how often the program can afford 125M?

Deliverable: One number and one sentence.

- (d) Peak memory. The throughput measurement at 30M peaked at 9.3 GB with $V = 32,000$. Two things in a step scale with V : the logits, BTV values in bf16 (2 bytes each), and the two $V \times d$ matrices, each with an fp32 weight, an fp32 gradient, and two fp32 Adam moments (16 bytes per parameter). Compute both at $V = 32,000$ and at $V = 50,304$ and predict today’s peak.

Deliverable: An expression of the form $9.3 - a + b$ with numerical a and b in GB, and the predicted peak.

6 Predictions

Before running any experiments, think about what the numbers should be. Two of the predictions can be derived rather than guessed, and the derivations are given here so that a miss is informative.

How to derive the final loss. Hoffmann et al. [J. Hoffmann et al., 2022, §3.3] fit the loss of a transformer to

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad E = 1.69, A = 406.4, B = 410.7, \alpha = 0.34, \beta = 0.28, \quad (6)$$

three pieces: a floor E that no model beats, a term that shrinks with parameters N , and a term that shrinks with training tokens D . It was fitted on a different tokenizer and corpus, so it is an anchor, not a law. A sanity anchor for its scale: at $N = 124\text{M}$ and $D = 10\text{B}$ it gives 3.06, while llm.c’s GPT-2 124M on 10B tokens of FineWeb (not the Edu subset) reaches 3.28 [A. Karpathy, 2024] Edu text is easier, so the formula sitting below the FineWeb number is the right way round.

Worked example for the 30M body, $N = 34.1 \times 10^6$, $D = 3 \times 10^8$: parameter term $\ln N = 17.34$, $\times 0.34 = 5.90$, $e^{-5.90} = 0.00275$, $\times 406.4 = 1.12$; token term $\ln D = 19.52$, $\times 0.28 = 5.47$, $e^{-5.47} = 0.00423$, $\times 410.7 = 1.74$; $L = 1.69 + 1.12 + 1.74 = 4.54$. Doing the same at 60M/600M and 125M/1.2B with body-only N gives 4.02 and 3.58. Then two judgment calls, to be written down with the prediction: (i) should N include the embeddings? At 30M the untied embedding plus head is $2 \times 50,304 \times 512 = 51.5\text{M}$ parameters, more than the body, and counting them gives 4.24, 3.81, 3.48; (ii) our batch of 32k tokens per step is small, which is slightly more token-efficient than the batches these fits used, so shade down a little.

How to derive the step-0 loss. Before training, the logits are small random numbers, so the model predicts a nearly uniform distribution over $V = 50,304$ tokens; the loss of a uniform guess is $\ln V = 10.826$. The random head adds a little. If each logit is $z_i \sim \mathcal{N}(0, v)$ independently, the loss at a position is $\log \sum_i e^{z_i} - z_y$. Since $\mathbb{E}[e^z] = e^{v/2}$, for large V the sum is close to $Ve^{v/2}$, so the loss is about $\ln V + v/2 -$

z_y , and $\mathbb{E}[z_y] = 0$. `fla` initializes the head from $\mathcal{N}(0, 0.02^2)$ and it acts on activations that the final normalization layer scales to unit root-mean-square, so $v = 0.02^2 d = 0.205$ at $d = 512$: add 0.10 and expect 10.93 at 30M and 10.98 at $d = 768$. A one-line check in the interpreter, `np.log(np.exp(np.random.randn(50304) * np.sqrt(v)).sum()) - np.log(50304)`, should be close to $v/2$. If a run reports 10.826 to three decimals, the head is zero-initialized, which is not wrong but changes this prediction.

Problem (predictions): Six numbers before the first run (1 point) [you]

Log these before proceeding.

- (a) The pooled seed standard deviation of final validation loss is the day's result, and the quantity every later comparison divides by. $\hat{s}_{\text{pooled}}$ in nats over three sizes at two seeds, the `s_pooled` field that `summarize.py` prints from your `seed_stats`; and separately $\hat{s}_{30\text{M}}$, tonight's single-pair number. The only published prior we found: modded-nanogpt, a heavily tuned 124M recipe on 0.9B FineWeb tokens, reports an inter-run standard deviation of up to 0.005 nats [K. Jordan and others, 2024]. Our recipe is untuned and our models smaller, so expect more.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry's prediction line.

- (b) The determinism floor tells us whether a seed pair measures the seed or the GPU. The largest absolute difference in logged training loss between the two same-seed smoke runs over 300 steps, in nats, from their `metrics.jsonl` files (Problem (smoke_floor)). Choose among: exactly 0, about 10^{-4} , about 10^{-2} , and say why. PyTorch documents the Flash backward as non-deterministic by default [PyTorch contributors, 2026].

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry's prediction line.

- (c) The final validation loss at each size is the number we will compare linear attention against. `final_val_loss` in `summary.json`, in nats, for 30M/300M, 60M/600M, and 125M/1.2B. Derive it from Equation 6 as above: body-only N gives 4.54, 4.02, 3.58 and total N gives 4.24, 3.81, 3.48. Give each with a $\pm$.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry's prediction line.

- (d) The step-0 validation loss is the first pipeline check: it tests the label shift and the evaluation code before any training has happened. The `val_loss` of the `kind = eval` row at `step = 0` in `metrics.jsonl`, in nats, at 30M. Derived above: $\ln V + v/2 = 10.93$ at $d = 512$.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry's prediction line.

- (e) Training throughput at 30M, relative to the measurement with the smaller vocabulary, checks the head-cost correction used in the budgets. The median `tok_s` over the `kind = train` rows of the 30M smoke run's `metrics.jsonl`, divided by 77,546. The head is $1.19 \times$ the matmul parameters at 30M (Section 4.1), so a naive bound is $1/1.19 = 0.84$; the head is the most efficient matmul in the model, so the truth is probably above that.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry's prediction line.

- (f) Whether the spread shrinks with model size decides whether pooling is fair to the small models. The direction of $\hat{s}_{30M}$, $\hat{s}_{60M}$, $\hat{s}_{125M}$ once all three pairs are in: yes (shrinks), no, or cannot tell at $n = 2$.

Deliverable: A value, a range, a direction where applicable, and a confidence (high / medium / low), in the wording of the log entry’s prediction line.

7 Experiments

Two pitfalls come before the script that meets them. First, timing: CUDA kernels are launched asynchronously, so the host clock read right after `loss.backward()` measures launch time, not run time. `train.py` calls `torch.cuda.synchronize()` at every logging point before reading the clock, and reports tokens per second over training steps only, with evaluation and checkpoint time subtracted, so that its `tok_s` field is comparable with the throughput measurements. Second, memory: the logits tensor is $BTV \times 2$ bytes = 3.3 GB in bf16 and does not shrink with the model; it is the largest object in the 30M run. The GB10’s unified memory means `nvidia-smi` reports N/A for memory, so the script logs `torch.cuda.max_memory_allocated()` as `peak_mem_GB` instead.

Controls. What varies across the six runs: the size (three values) and the seed (two values). What is held fixed, and why: every value in [Table 3](#), so that the two runs at a size differ in nothing but the seed and the three sizes differ in nothing but shape and budget; the window set at a size, so that “which text” is not part of the spread; and the validation windows, which are the same for every run and are never trained on (Problem (`check_disjoint`)). What the seed controls: the initial weights and the window order. What it does not control: the GPU’s summation order, which Problem (`smoke_floor`) measures with zero seed changes. The validation set may be looked at freely; it decides nothing today, since nothing is tuned.

Algorithm 1: One training run, as `train.py` executes it

```

1 Require: a run file (attn_<size>.toml), the seed
2  $S \leftarrow \lfloor D/(BT) \rfloor$ ; build the model with torch.manual_seed(seed); AdamW on two
  parameter groups
3  $order \leftarrow \text{batch\_order}(seed, S * B)$ , a permutation of the first  $SB$  training
  windows
4 evaluate on the 4096 validation windows; log as step 0
5 for  $s = 0, \dots, S - 1$  do
6    $x \leftarrow \text{windows } order[s*B : (s+1)*B]$ , int64 ( $B, T$ )
7   set the learning rate to lr_at(s, S)
8    $\ell \leftarrow \text{model}(\text{input\_ids}=x, \text{labels}=x).loss$  under bf16 autocast and the SDPA
    Flash context
9   backward; clip the gradient norm to 1.0; AdamW step; zero gradients
10  every 20 steps: synchronize, log mean loss, learning rate, gradient norm, tokens
    per second, peak memory
11  every 5% of  $S$ : evaluate on the 4096 windows; every 10%: write checkpoints
12 end for
13 evaluate on the 16,384 windows; write summary.json
```

Problem (train_script): The training script (0.5 points) [AI]

`scripts/train.py` implements [Algorithm 1](#). Its inputs are `--config` (a run's TOML file), `--seed`, and an optional `--out` directory; every other value, including the token budget and the `training.steps` key that stops a smoke run early while keeping the full-budget schedule, is read from the file, so the flags of a run are its file and never the command line. It imports your `lr_at` and `evaluate`. It skips a run whose `summary.jsonl` exists and resumes a run whose `ckpt_latest.pt` exists. It raises on a non-finite loss.

It writes the run directory `runs/<mixer>_<size>_s<seed>/`, or the `--out` directory, with `config.toml` (a verbatim copy of the file it was launched with), `env.json` (the versions of `torch`, `fla`, and `triton`, the GPU kernel compiler `fla` builds on; the GPU name; and a hash of the `testbed/` and `scripts/` sources), `metrics.jsonl`, the checkpoints, and `summary.json` at the end; `run.sh` adds `stdout.log`.

Metrics it logs, in `metrics.jsonl`: the training loss averaged over each 20-step interval, the learning rate, the mean gradient norm, tokens per second, peak allocated memory, wall-clock seconds, and the validation loss at every evaluation. It logs no qualitative samples, since a loss baseline has none to show; if you want to see text, decode a validation window with `look_at_shards.py`.

Deliverable: The script, shipped. Your part is to read the loop once, top to bottom, before running it.

Debugging Tip: Three checks, in order, before trusting any curve

In `metrics.jsonl`: the step-0 validation loss is within 0.05 nats of your prediction (otherwise `evaluate` or the label shift is wrong); the training loss at step 300 is well below 7.5 (stuck near 10.8: `lr_at` returns 0 or is not applied; falling toward 0: labels are not shifted); tokens per second is within 15% of your throughput prediction (otherwise the loader stalls, or evaluation time is leaking into the timed window). Our estimate for the step-300 loss at 30M is between 6 and 7 nats: warmup ends at step 275, so the run has had a few dozen steps at the peak rate.

Problem (smoke_floor): Smoke test and the determinism floor (0.2 GB10 hrs) (1 point) [you]

Resource requirements: ≤ 15 minutes of GPU for two runs, ≤ 12 GB GPU memory

- (a) Warm up the analysis script on the shipped synthetic runs before there are real ones, so that you know what its output looks like:

```
$ head -c 230 experiments/day3_seed_variance/sample_runs/attn_30M_s0/metrics.jsonl;
echo; sed
↵ -n 2p experiments/day3_seed_variance/sample_runs/attn_30M_s0/metrics.jsonl
{"kind": "meta", "mixer": "attn", "size": "30M", "seed": 0, "d": 512, "L": 10,
"heads": 8,
↵ "vocab": 50304, "params_M": {"body_M": 34.1, "embed_M": 25.8, "head_M": 25.8,
"total_M":
↵ 85.6}, "budget_tokens": 300000000, "steps_full": 9 ...
```

```
{ "kind": "eval", "step": 0, "tokens": 0, "val_loss": 10.92768490070576,
  "val_windows": 4096,
  "eval_s": 18.0}
$ python scripts/summarize.py --runs experiments/day3_seed_variance/sample_runs
```

The first record of every log is the meta line (one JSON object per line, UTF-8; `params_M` in millions; `val_loss` in nats); the second is the step-0 evaluation. The four runs are written by a formula and marked `"synthetic": true`, and their `config.toml` and `env.json` are placeholders; their numbers mean nothing.

Deliverable: The table `summarize.py` prints, and one sentence on why its pooled ν is 2 and not 3.

- (b) Run the smoke pair. Each is the first 300 steps of the real seed-0 run, same order and same schedule:

```
python scripts/train.py --config experiments/day3_seed_variance/attn_30M_smoke.toml
--seed 0
↪ --out runs/smoke_a
python scripts/train.py --config experiments/day3_seed_variance/attn_30M_smoke.toml
--seed 0
↪ --out runs/smoke_b
```

The smoke file is `attn_30M.toml` with `training.steps = 300`; nothing else differs. Apply the Debugging Tip's three checks to `runs/smoke_a/metrics.jsonl`, and give the evidence that convinced you that the script is correct. **Note:** It is normal for the training loss to sit near 10.8 for the first twenty or so steps, while the learning rate is still tiny, and for the gradient norm to be above 1 early so that clipping is active.

Deliverable: Three numbers (step-0 validation loss, step-300 training loss, median tokens per second) with a pass/fail each, and two sentences of evidence. Grade the step-0 and throughput predictions.

- (c) Compare `loss` at every logged step between the two runs.

Deliverable: Either the word “bitwise”, or $\max_t |\Delta_t|$ in nats and one sentence on whether it grows with t . Grade the determinism-floor prediction. If the floor exceeds 0.02 nats, the pivot in the overview applies.

- (d) Compare the measured `peak_mem_GB` with the prediction of Problem (budgets) (d).

Deliverable: The measured number and one sentence on the gap.

Problem (seed_pairs): The day's result (2.4 GB10 hrs today; about 28 more for the queue) (2 points) [you]

Resource requirements: ≤ 2.5 hours of GPU today for the 30M pair, ≤ 12 GB GPU memory; the 60M and 125M pairs continue for about 28 hours and need ≤ 25 GB

- (a) Delete `runs/smoke_a` and `runs/smoke_b`, so that `summarize.py` reads only the queue's runs, and launch the queue: `nohup experiments/day3_seed_variance/run.sh > run.log 2>&1 &`. The order, from `sweep.py`: 30M seed 0, 30M seed 1, 60M seed 0, 60M seed 1, 125M seed 0, 125M seed 1. Each run writes `runs/attn_<size>_s<seed>/metrics.jsonl` as it goes, so `python`

`scripts/summarize.py` works at any time. If the box dies, re-launch the same command: finished runs are skipped and the interrupted run resumes from its last checkpoint. The 30M pair should finish about 2.5 hours after launch by the estimate of Problem (budgets); if a run is much slower, check `tok_s` in its log against the smoke run first, and then whether data preparation or another process is sharing the GPU.

Deliverable: Pipeline health, when `runs/attn_30M_s1/summary.json` exists: how many of the two runs failed or restarted (from `run.log`), and whether any `kind = train` row has a non-finite loss or a gradient norm above 10.

- (b) How fast was it.

Deliverable: The median `tok_s` of each 30M run, compared with the smoke run and with $77,546 \times$ your throughput prediction.

- (c) The metric. Run `python scripts/summarize.py`.

Deliverable: The table, with x_1 , x_2 , $\hat{s}_{30M}$, and the $\nu = 1$ range; and `experiments/day3_seed_variance/baselines.png`, which `summarize.py` writes and in which each seed is its own curve (the spread is shown as the two individual runs, not as a mean).

- (d) Read the curves.

Deliverable: Two sentences: where the two seed curves separate most, and whether they reconverge during the final decay.

- (e) Pass bar for the pipeline: both runs complete with a finite loss at every logged step; the step-0 validation loss of each is within 0.05 nats of the prediction; and the final 30M validation loss is below 4.8 nats, a loose bound above the 4.24 to 4.54 range that Equation 6 gives, so that a broken schedule or a wrong loss average is caught while a merely surprising spread is not. Then grade the seed-spread prediction for 30M.

Deliverable: Pass or fail on each of the three, and the graded prediction line. The 60M and 125M pairs, the pooled $\hat{s}$, the $\nu = 3$ range, the MDD, and the remaining predictions are graded when the queue finishes; add them to the same log entry.

- (f) Given the variance between the two runs, how confident are you in $\hat{s}_{30M}$?

Deliverable: One sentence, in the log entry.

8 What to observe

Tonight's numbers. x_1 and x_2 at 30M; $\hat{s}_{30M}$ from Equation 1; the $\nu = 1$ range from Table 1, which is wide, and the width is the point; the determinism floor and the evaluation noise, both of which should be well below $\hat{s}$. If the floor is comparable to $\hat{s}$, you measured the GPU, not the seed, and that is itself the result.

The plot. Two seed curves at 30M. Typically they are far apart early, when the initialization dominates, converge in the middle, and either collapse together or re-separate in the final decay. The final gap is what you record; the trajectory tells you whether the “settling” story of Section 4.3 holds.

Surprises worth a paragraph. $\hat{s}_{30M} > 0.03$: the first things to look at are whether lr in the log is decaying and whether there is a mid-run spike, and only if both are clean does the conclusion become that 30M at this learning rate is that noisy; if so, 30M rows in later comparisons will be uninformative, and the question of the next day becomes whether the baseline recipe is stable at 30M. $\hat{s}_{30M} < 0.003$: the things to look at are whether `seed` differs in the two `meta` lines and whether the first batches differ, and then how the number compares with the floor. Step-0 loss far from 10.93: the label shift or `evaluate` is

wrong; fix before anything else. Tokens per second more than 15% below prediction: loader stall, or evaluation time leaking into the timed window.

A null result is a result. If the pooled $\hat{s}$ turns out below 0.005 nats, the band is narrower than the published prior and later comparisons can detect small effects; if it is above 0.03, most published small-scale architecture claims at this scale are inside the band, and saying so loudly is the finding. Neither outcome is a failure of the day.

Broken pipeline, not a result. A non-finite loss; a loss falling toward 0; `check_disjoint` returning a large count; two same-seed runs that differ at step 0 (the initialization is not seeded).

The pooling assumption. The band assumes σ is the same at all three sizes. If larger models wobble less, pooling is conservative for 125M and anti-conservative for 30M. This is why the log entry carries the per-size and the pooled numbers side by side (Problem (`log_entry`)), and why a third seed at whichever size a later comparison lands near the band is candidate (B) in Problem (`your_call`).

Problem (`your_call`): Tomorrow's follow-up (0.5 points) [you]

Two candidate follow-ups, each one day: (A) split the seed's effect into initialization and data order with a 2×2 crossing at 30M, two initializations by two orders, four runs of about 1.2 h each; (B) add a third seed at every size, three runs of about 15 GPU-hours in total, which moves the pooled ν from 3 to 6 (Table 1) and is the only way to say whether σ shrinks with size.

(a) Choose one and argue for it.

Deliverable: A few sentences, in the log entry.

Problem (`log_entry`): Experiment log (0.5 points) [you]

Fill in the entry below. It is identical to `experiments/day3_seed_variance/log_entry_template.md`. The “What I now believe” paragraph must contain the sentence “with two seeds per arm this program cannot detect a loss difference smaller than about ____ nats”, and a sentence on how much you trust that number given one pair tonight and three later.

```
### Day 3, 2026-09-15
Question: How much does the random seed alone move a small transformer's final
validation
↳ loss, at 30M/300M tokens today and at 60M/600M and 125M/1.2B in the queue?
Prediction (written BEFORE running): what number do I expect, and how confident?
Setup: attention baseline (fla TransformerForCausalLM, SDPA Flash), 30M/60M/125M
↳ (d=512/768/768, L=10/9/18), GPT-2 tokenizer, vocab 50304; AdamW 6e-4 peak, 3% warmup,
cosine
↳ to 0.1x, B=32, T=1024; budgets 300M/600M/1.2B tokens; seeds {0, 1}; the seed sets the
init
↳ and the order of a fixed window set; eval on the first 2^24 tokens of shard 0
Result: (one plot/table, path in repo)
Prediction correct? (yes / no / partially, and what surprised me)
What I now believe:
Spawned question(s):
Hours spent:
```

Deliverable: The filled-in entry, with the per-size and the pooled numbers both on the result line, the predictions graded item by item, a “noticed but not chased” line for anything odd you saw in the logs and did not pursue, and your call from Problem (`your_call`).

9 Things that are easy to get wrong

- Quoting a two-run standard deviation as if it were exact. It can be off by a factor of several either way ([Table 1](#)); the number is only honest pooled across sizes and with its range beside it.
- Forgetting that the band is tied to the schedule. The final low-learning-rate stretch pulls runs together ([Section 4.3](#)). Reading loss mid-training, or under a constant learning rate, needs a fresh band.
- Assuming loss spread equals accuracy spread. Recall accuracy on a synthetic task at this scale can swing by tens of points across seeds; today’s band says nothing about it, and that evaluation will need its own seeds.
- Treating “seed” as one thing. Initialization and data order can be separated with a 2×2 crossing; GPU non-determinism is measured with zero seed changes. Today measures the sum; say which piece you mean when you use the number.
- Forgetting that the 30M model is more than half embedding. Untied $50,304 \times 512$ in and out is 51.5M parameters against a 34.1M body. “30M” is the body; any scaling-law fit must say which N it uses.
- Editing a file while the queue is running. Each run is a fresh process that re-imports the package from disk, so a file edited mid-queue changes the later runs and not the earlier ones, and the six runs stop being one configuration.

Problem (`explain_it_back`): Explain it back (0.5 points) [you]

- (a) In four to six sentences, in your own words and without looking at [Section 3](#): what number did today produce, what does it mean for a claim like “the linear-attention model is 0.02 nats worse than attention at 60M”, why can one pair of runs not pin the number down, and what did the same-seed rerun rule out?

Deliverable: A few sentences.

References

- [1] PyTorch contributors, “Reproducibility.” 2026.
- [2] A. Karpathy, “llm.c: LLM training in simple, raw C/CUDA.” 2024.
- [3] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models.” 2022.
- [4] K. Jordan, others, “modded-nanogpt: NanoGPT (124M) in 5 minutes.” 2024.